Injection Attacks in LLMs

Horus Project Updates - April 29, 2026

Ana Clara Zoppi Serpa

Contents

Recap - Status When I Last Presented

Status Recap

- We had discussed the vulnerability of LLMs to **Injection Attacks**.
- It became clear that we need a deeper understanding of **how Transformers process prompts** internally.

Decision

To crack the “black box” and investigate these internal pathways, I decided to dive into **Mechanistic Interpretability**.

News! ✨

What I've been doing

Studying Fundamentals

- Completed *Essence of Linear Algebra* [1].
- Followed Neel Nanda's guide [2].
- Read the Mech Interp glossary [3].

Hands-on Practice

- Started the **ARENA** exercises [4].
- Digging into Anthropic's *Mathematical Framework for Transformer Circuits* [5].

Mechanistic Interpretability

What and Why?

What is it?

Taking a trained model and **reverse engineering** the algorithms it learned during training from its weights.

Why does it matter?

We have programs that speak English at human levels, but we have *no idea* how they work!

Understanding this is crucial for **AI Safety & Alignment**.

Relationship with Injection Attacks

- Injection attacks **hijack** model behavior by manipulating inputs.
- Mech Interp can reveal exactly **how** and **where** these attacks manipulate the model's internal representations.

Potential to detect or block malicious paths directly inside the **residual stream**.

Transformer Model & Linear Algebra

The Residual Stream

- A high-dimensional vector space acting as a **communication channel** [5].
- No “privileged basis” — information is added and persists unless actively deleted.
- Components read from and write to different subspaces.
- Can be decomposed into paths via **virtual weights**.

Residual stream bandwidth is in very high demand — layers communicate in superposition!

Attention Heads vs MLP Layers

Attention Heads

- Move information **between positions**.
- Independent operations that write into the residual stream.

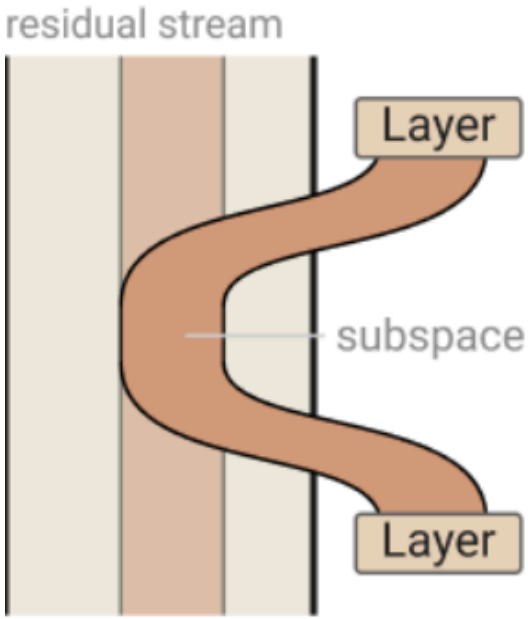
MLP Layers

- Process information independently **at each position**.
- “Thinking” after grabbing info via attention.

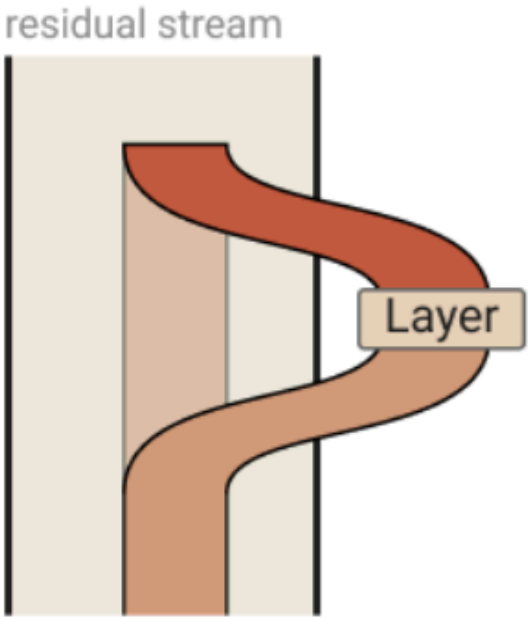
Relevant Snippets from the Paper

ttention heads can be understood as having two largely independent computations: a
QK ("query-key") circuit which computes the attention pattern, and an OV ("output-
alue") circuit which computes how each token affects the output if attended to.

Key, query, and value vectors can be thought of as intermediate results in the
computation of the low-rank matrices $W_Q^T W_K$ and $W_O W_V$. It can be useful to
describe transformers without reference to them.



Layers can interact by writing to and reading from the same or overlapping subspaces. If they write to and read from disjoint subspaces, they won't interact. Typically the spaces only partially overlap.



Attention Heads are Independent and Additive

As seen above, we think of transformer attention layers as several completely independent attention heads $h \in H$ which operate completely in parallel and each add their output back into the residual stream. But this isn't how transformer layers are typically presented, and it may not be obvious they're equivalent.

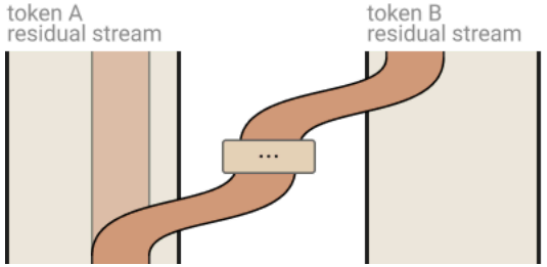
In the original Vaswani *et al.* paper on transformers [1], the output of an attention layer is described by stacking the result vectors $r^{h_1}, r^{h_2}, \ldots$, and then multiplying by an output matrix W_O^H. Let's split W_O^H into equal size blocks for each head $[W_O^{h_1}, W_O^{h_2} \ldots]$. Then we observe that:

$$W_O^H \begin{bmatrix} r^{h_1} \\ r^{h_2} \\ \ldots \end{bmatrix} = \left[W_O^{h_1},\ W_O^{h_2},\ \ldots\right] \cdot \begin{bmatrix} r^{h_1} \\ r^{h_2} \\ \ldots \end{bmatrix} = \sum_i W_O^{h_i} r^{h_i}$$

Revealing it to be equivalent to running heads independently, multiplying each by its own output matrix, and adding them into the residual stream. The concatenate definition is often preferred because it produces a larger and more compute efficient matrix multiply. But for understanding transformers theoretically, we prefer to think of them as independently additive.

Attention Heads as Information Movement

But if attention heads act independently, what do they do? The fundamental action of attention heads is moving information. They read information from the residual stream of one token, and write it to the residual stream of another token. The main observation to take away from this section is that which tokens to move information from is completely separable from what information is "read" to be moved and how it is "written" to the destination.



Attention heads copy information from the residual stream of one token to the residual stream of another. They typically write to a different subspace than they read from.

A major benefit of rewriting attention heads in this form is that it surfaces a lot of structure which may have previously been harder to observe:

- Attention heads move information from the residual stream of one token to another.
    - A corollary of this is that the residual stream vector space — which is often interpreted as a "contextual word embedding" — will generally have linear subspaces corresponding to information copied from other tokens and not directly about the present token.
- An attention head is really applying two linear operations, A and $W_O W_V$, which operate on different dimensions and act independently.
    - A governs which token's information is moved from and to.
    - $W_O W_V$ governs which information is read from the source token and how it is written to the destination token.[9]

Residual stream

The residual stream is the sum of all previous outputs of layers of the model, and is also the input to each new layer. It has shape `[batch, seq_len, d_model]` (where `d_model` is the length of a single embedding vector).

The initial value of the residual stream is denoted x_0 in the diagram, and x_i are later values of the residual stream (after more attention and MLP layers have been applied to the residual stream).

The residual stream is *really* fundamental. It's the central object of the transformer. It's how model remembers things, moves information between layers for composition, and it's the medium used to store the information that attention moves between positions.

▼ Aside - **logit lens**

A key idea of transformers is the residual stream as output accumulation. As we move through the layers of the model, shifting information around and processing it, the values in the residual stream represent the accumulation of all the inferences made by the transformer up to that point.

This is neatly illustrated by the **logit lens**. Rather than getting predictions from the residual stream at the very end of the model, we can take the value of the residual stream midway through the model and convert it to a distribution over tokens. When we do this, we find surprisingly coherent predictions, especially in the last few layers before the end.

TransformerLens & Tools

- Library by Neel Nanda for Mech Interp of GPT-2 style models [6].
- Key goal: **Exploratory Analysis**.
- Keeps the feedback loop between idea and result extremely short!

Core Operations

- **Hooks**: Access and intervene on activations.
- **Logit Attribution**: Decompose output logits into terms from each component.

Hooks & Interventions

- Every activation inside the transformer is surrounded by a **hook point**.
- We can access intermediate states (e.g., `run_with_cache`).
- We can intervene via **ablation**: modifying values on the fly to see how the model's predictions change.

Hook points act on *activations* (not layers), inspired by Garçon [7].

In-Context Learning & Circuits

In-Context Learning (ICL)

- The ability of a model to learn a new task **during inference** (from the prompt).
- **No weight updates** happen.
- Implemented internally via specific circuits, primarily **Induction Heads** [8].

$$P(y \mid x, C)$$

— the output y is conditioned not just on input x, but on the context C in the prompt.

Induction Heads

- A mechanistic circuit for **copying patterns** [8].
- If token `B` followed token `A` previously, it predicts `B` will follow `A` again.
- Evaluated by seeing if it detects patterns like `A B C D A B`.

Induction heads are the mechanistic proof that In-Context Learning is not magic — it is **computation localised in specific attention heads**.

Circuits & Reverse Engineering

- **Circuit**: Interpretable end-to-end functions mapping tokens to changes in logits [5]. They correspond to “paths” through the model.
- **Reverse Engineering**: Deducing human-understandable programs embedded within the neural network.

Superposition makes this challenging — residual stream bandwidth is highly contested [9].

Resources & References

Learning Resources & Tools

Foundational Learning

- ARENA [4]
- Essence of Linear Algebra [1]
- Mech Interp Research Guide [2]
- Mech Interp Glossary [3]

Tools

- TransformerLens [6]
- CircuitsVis [10]

Anthropic's Transformer Circuits Thread

- A Mathematical Framework for Transformer Circuits [5]
- In-Context Learning and Induction Heads [8]
- Toy Models of Superposition [9]
- Visualizing Weights (Garçon) [7]

Open Problems

- 200 Concrete Open Problems in Mechanistic Interpretability [11]

References

[1] G. Sanderson, “Essence of Linear Algebra.” [Online]. Available: https://www.3blue1brown.com/topics/linear-algebra

[2] N. Nanda, “How to Become a Mechanistic Interpretability Researcher.” [Online]. Available: https://www.alignmentforum.org/posts/jP9KDyMkchuv6tHwm/how-to-become-a-mechanistic-interpretability-researcher

[3] N. Nanda, “Mechanistic Interpretability Glossary.” [Online]. Available: https://www.neelnanda.io/mechanistic-interpretability/glossary

[4] C. McDougall, “ARENA: Alignment Research Engineer Accelerator.” [Online]. Available: https://arena-chapter1-transformer-interp.streamlit.app/

[5] N. Elhage *et al.*, “A Mathematical Framework for Transformer Circuits,” 2021, [Online]. Available: https://transformer-circuits.pub/2021/framework/index.html

[6] N. Nanda, “TransformerLens.” [Online]. Available: https://github.com/neelnanda-io/TransformerLens

[7] T. Conerly, N. Elhage, and C. Olah, “Visualizing Weights (Garçon),” 2021, [Online]. Available: https://transformer-circuits.pub/2021/garcon/index.html

[8] C. Olsson *et al.*, “In-Context Learning and Induction Heads,” 2022, [Online]. Available: https://transformer-circuits.pub/2022/in-context-learning-and-induction-heads/index.html

[9] N. Elhage *et al.*, “Toy Models of Superposition,” 2022, [Online]. Available: https://transformer-circuits.pub/2022/toy_model/index.html

[10] TransformerLens Org, “CircuitsVis.” [Online]. Available: https://github.com/TransformerLensOrg/CircuitsVis

[11] N. Nanda, “200 Concrete Open Problems in Mechanistic Interpretability.” [Online]. Available: https://www.alignmentforum.org/posts/LbrPTJ4fmABEdEnLf/200-concrete-open-problems-in-mechanistic-interpretability